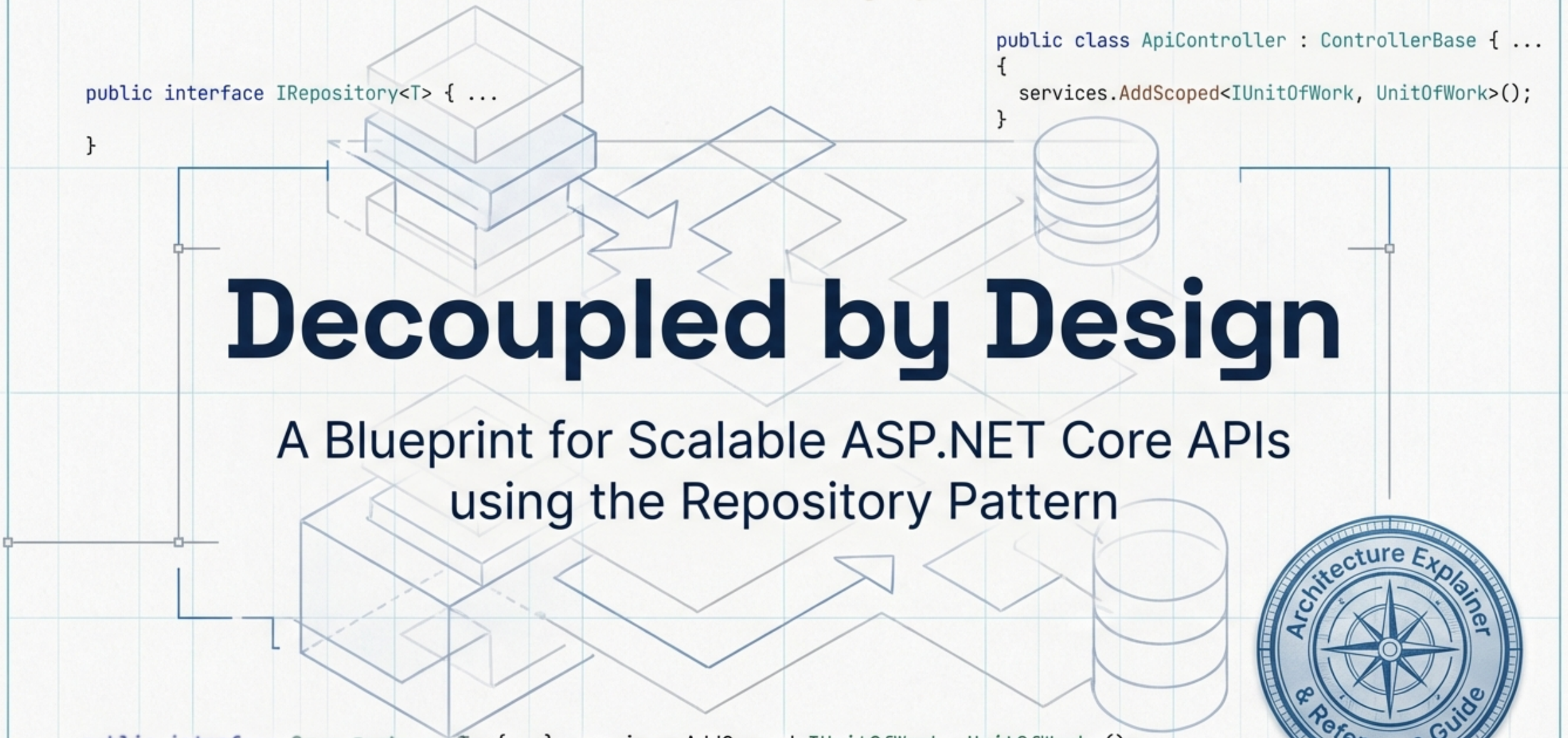
```
await _context.SavaEChangemSync();
```

```
public class ApiController : ControllerBase { ...
{
  services.AddScoped<IUnitOfWork, UnitOfWork>();
}
```

```
public interface IRepository<T> { ...

}
```



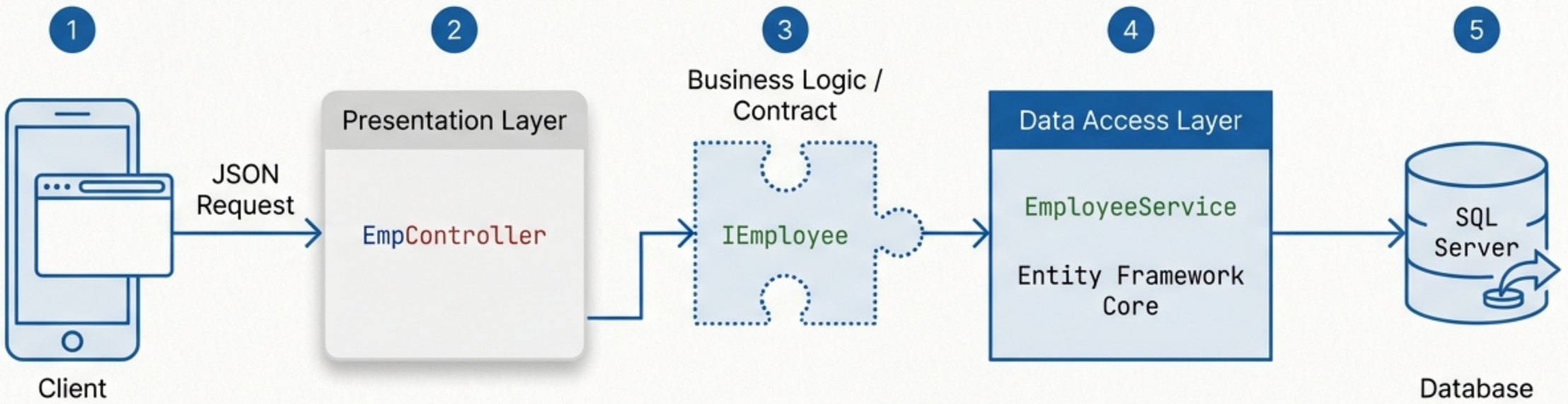
Decoupled by Design

A Blueprint for Scalable ASP.NET Core APIs using the Repository Pattern

```
public interface SavevEnctancy<T> {...}  services.AddScoped<IUnitOfWork, UnitOfWork>();
```

NotebookLM

The Architecture of an API Request



Data flows linearly. Dependencies point inward. The Controller never touches the Database directly.

NotebookLM

Why Overcomplicate? The Case for the Repository Pattern

	Direct DB Access (Tightly Coupled)	Repository Pattern (Decoupled)
Coupling	Controller is chained to **Entity Framework**.	Controller only knows about a generic **Interface**.
Testability	Requires a live SQL **database** to test API logic.	Easily **mockable** data for instant unit testing.
Code Duplication	Same LINQ queries **copy-pasted** across endpoints.	Centralized **data logic** in one service class.
Separation of Concerns	API manages web traffic **AND** database translation.	**Distinct layers** for distinct jobs.

NotebookLM

Step 1: Laying the Foundation with Data Models

```
using System.ComponentModel.DataAnnototions;

namespace WebApiInAsp.netcoreMvcDemo.Models
{
    public class Employee {
        public int Id { get; set; }
        // Additional properties inferred
    }
}
```

Required for Entity Framework to understand schema constraints.

EF Core automatically recognizes 'Id' as the Primary Key for the SQL table.

NotebookLM

Step 2: Equipping the Environment

`Microsoft.EntityFrameworkCore`

The core Object-Relational Mapper (ORM) engine.

`Microsoft.EntityFrameworkCore.SqlServer`

The specific database provider that translates C# into SQL Server syntax.

`Microsoft.EntityFrameworkCore.Tools`

Enables command-line tools for running database migrations.

NotebookLM

Step 3: Bridging Objects and Tables with DbContext

```
using Microsoft.EntityFrameworkCore;

namespace WebApiInAsp.netcoreMvcDemo.Models {
  public class EmpContext : DbContext {
    public EmpContext(DbContextOptions
       dbContextOptions) : base(dbContextOpti
    public DbSet<Employee> employees { get;
  }
}
```

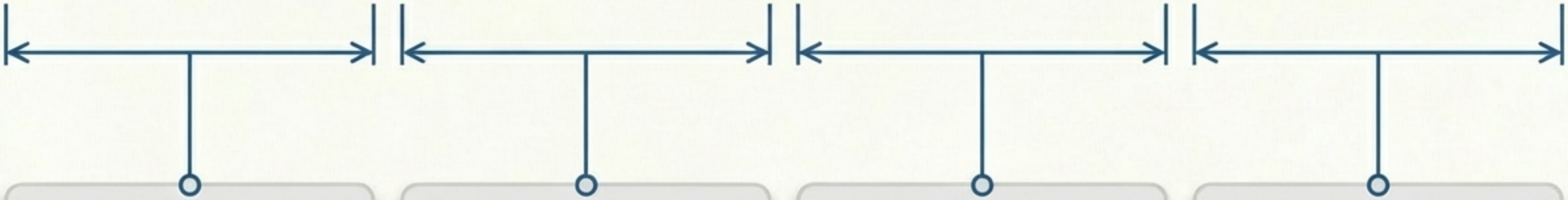
Inheriting Entity Framework's base functionality.

Accepts configuration (like connection strings) from the application startup.

NotebookLM

Dissecting the Connection String

```
Data Source=LAPTOP-4G8BHPK9\SQLEXPRESS;initial
catalog=EmpCg;Integrated Security=true;Encrypt=true;
TrustServerCertificate=true;
```



The Host: Points to the specific local SQL Server instance.

The Database: The exact name of the target database.

Authentication: Uses Windows credentials instead of a hardcoded SQL username/password.

Security Constraints: Forces encrypted connections but bypasses SSL certificate validation for local development.

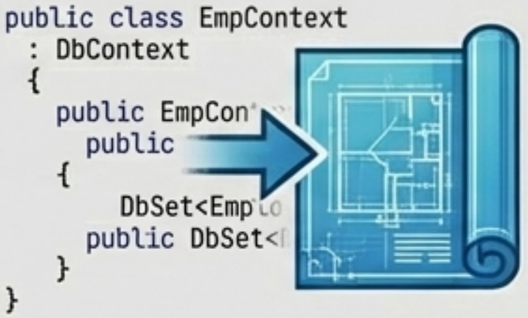
NotebookLM

Materializing the Blueprint: EF Core Migrations

1. Build

Build the solution to ensure no compilation errors.

2. Migrate



Run EF Core Migrations to generate the SQL schema based on the EmpContext.

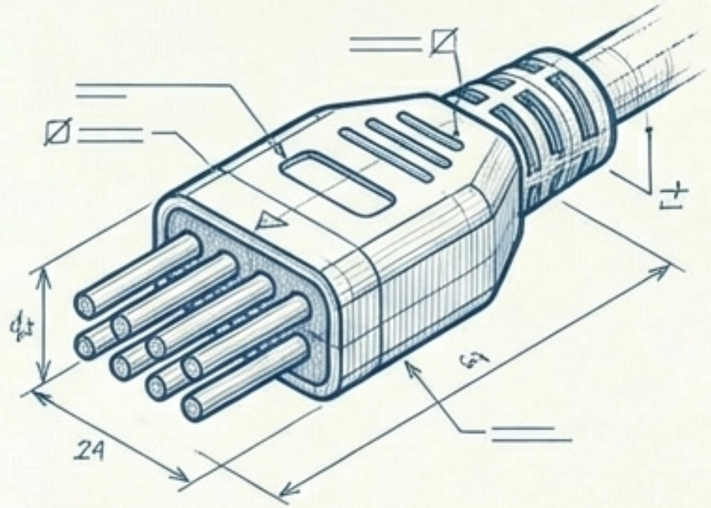
3. Seed

Add dummy data into the newly created **employees** table via SQL Server.

NotebookLM

Step 4: Forging the Contract with an Interface

IEmployee



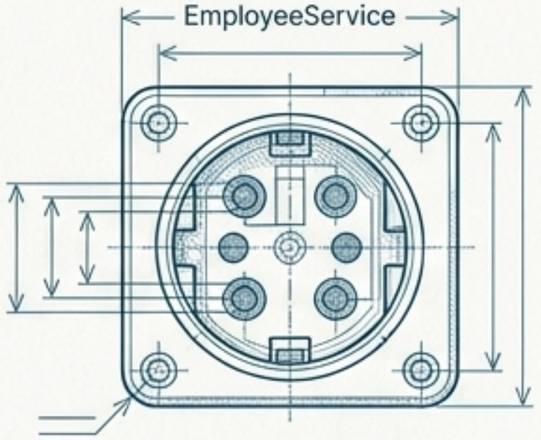
The Controller only cares about the shape of this plug, not what it connects to.

```
public interface IEmployee
{
    Task<List<Employee>> GetAllEmployeesAsync();
    Task<Employee?> GetEmployeeByIdAsync(int id);
    Task<Employee> AddEmployeeAsync(Employee employee);
    Task<Employee?> UpdateEmployeeAsync(Employee employee);
    Task<Employee?> DeleteEmployeeAsync(int id);
}
```

A strict contract defining **WHAT** the system can do, without dictating **HOW** it does it.

NotebookLM

Step 5: Implementing the Repository Logic




```
public class EmployeeService : IEmployee {
    private readonly EmpContext _context;

    public EmployeeService(EmpContext context) { _context = context; }

    public async Task<Employee> AddEmployeeAsync(Employee employee) {
        await _context.employees.AddAsync(employee);
        await _context.SaveChangesAsync();
        return employee;
    }
}
```

Fulfilling the interface contract.

Injecting the database context.

Executing the actual SQL transaction.

NotebookLM

Step 6: Exposing the System via the API Controller

```
[Route("api/[controller]")]
[ApiController]
```

Configures the web URL endpoint (e.g., /api/emp).

```
private readonly IEmployee _employeeService;
public EmpController(IEmployee employeeService) {
    _employeeService = employeeService;
}
```

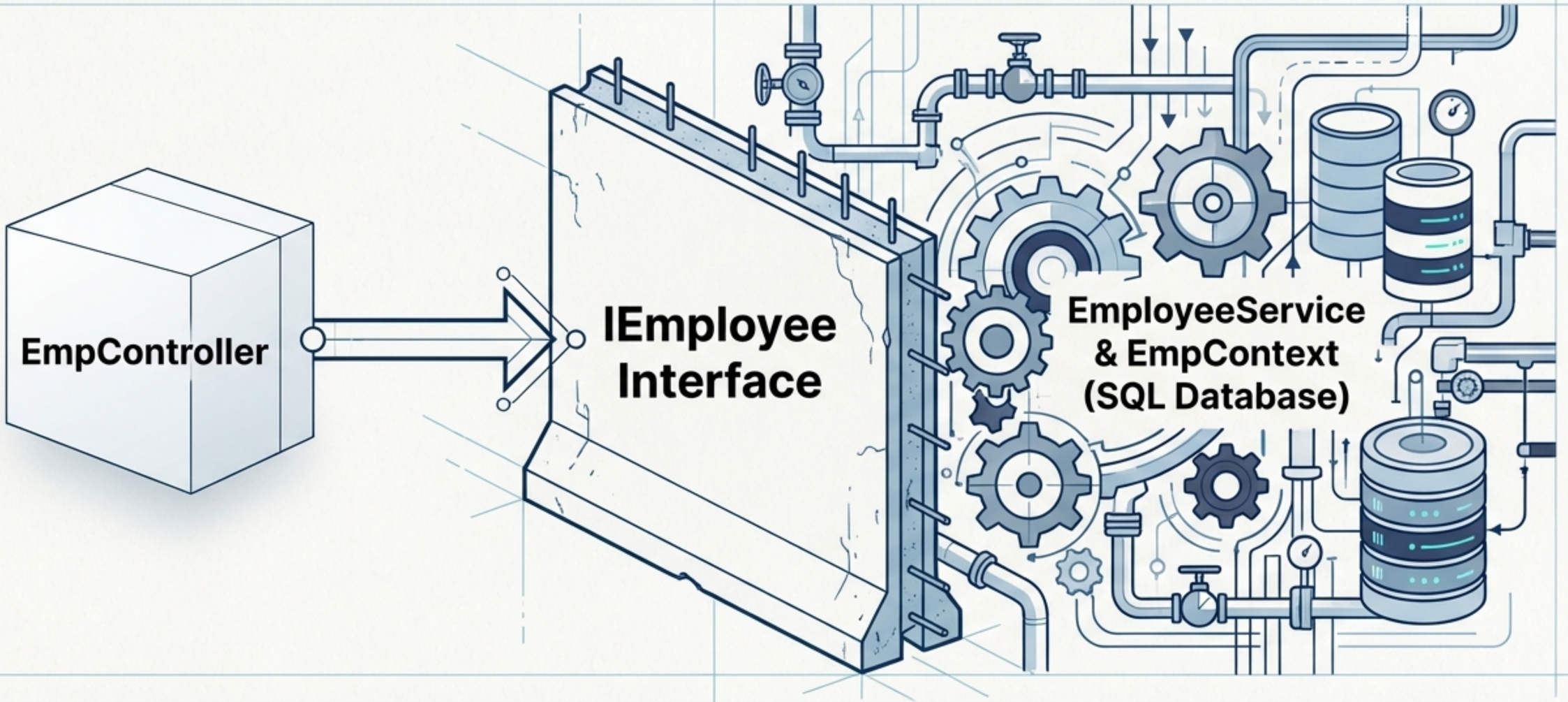
Injects the Interface (IEmployee), strictly avoiding any direct reference to the implementation (EmployeeService).

```
[HttpGet]
public async Task<ActionResult<List<Employee>>> GetAll() {
    return Ok(await _employeeService.GetAllEmployeesAsync());
}
```

Handles the HTTP request and returns an HTTP 200 (OK) with the data.

NotebookLM

The Power of the Blind Spot



Because the Controller only communicates with the **IEmployee** interface, you can completely swap out the underlying database (e.g., SQL Server to PostgreSQL) without changing a single line of Controller code.

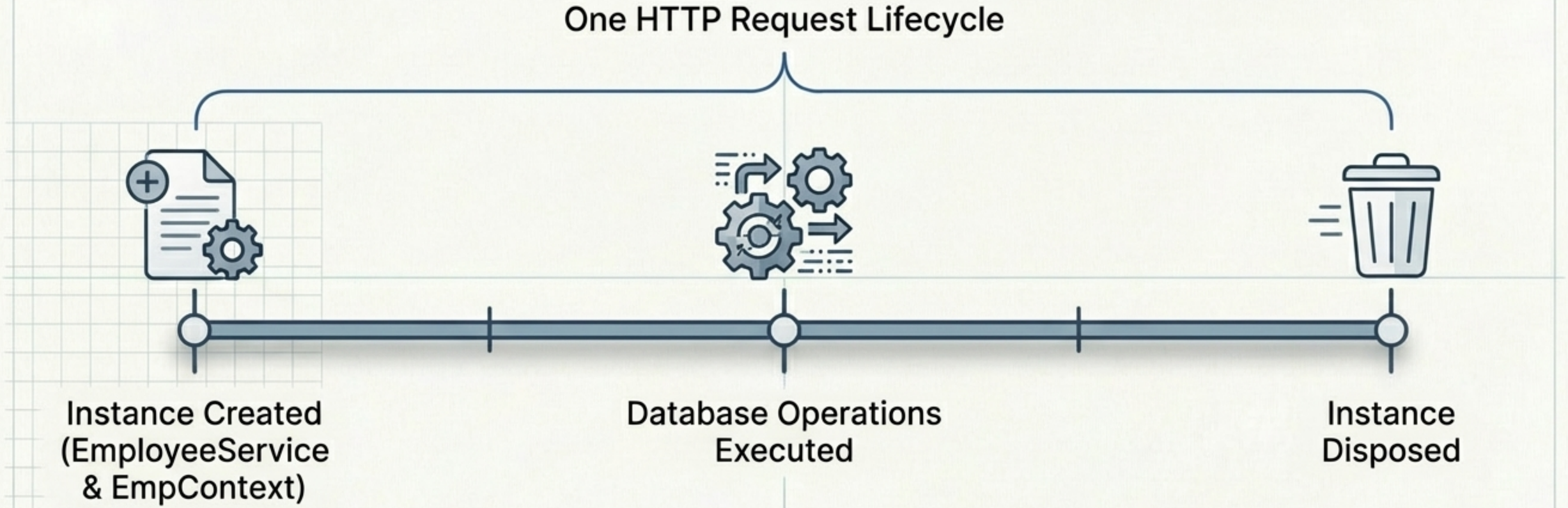
NotebookLM

The Central Brain: Dependency Injection

This single line of code is the glue of the entire architecture. It tells the application: Whenever a class asks for the **IEmployee** plug, hand them the **EmployeeService** socket.

NotebookLM

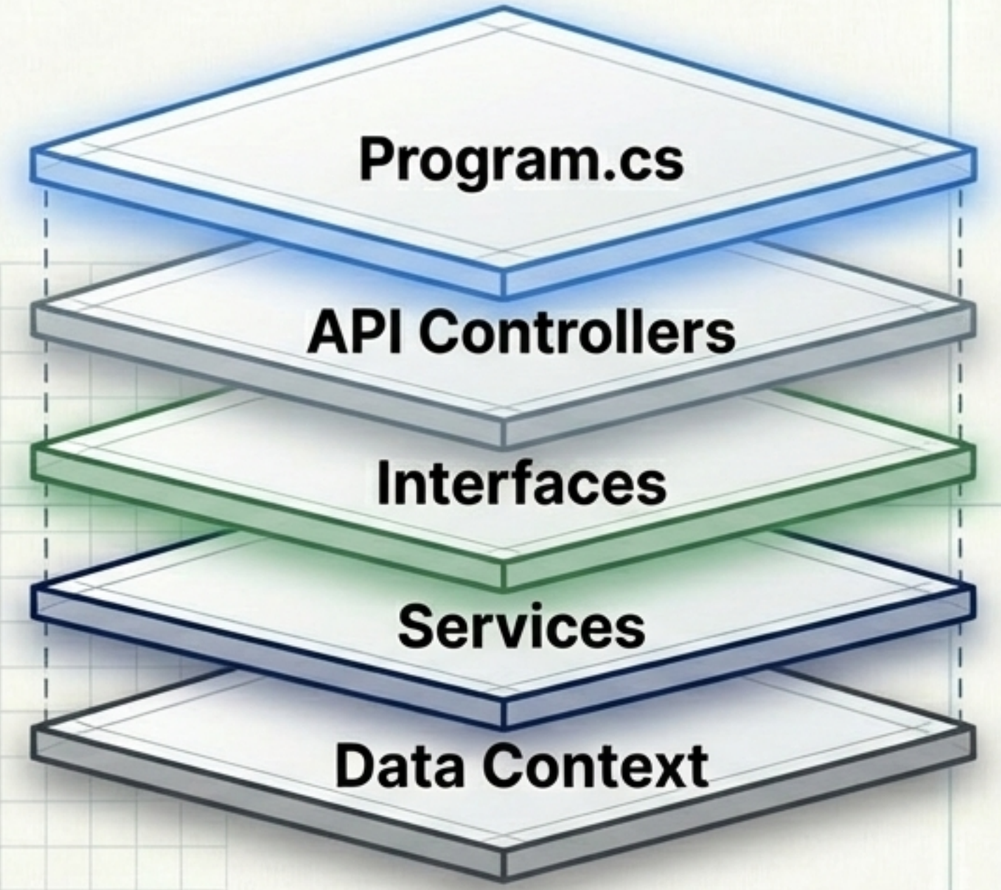
Defining the Lifespan: AddScoped



By registering the service as Scoped, .NET guarantees that a single, shared instance of **EmployeeService** (and its database context) is created at the start of an HTTP request and safely destroyed when the request ends. This prevents memory leaks and ensures data consistency.

NotebookLM

The Completed Architecture



The wiring (**AddScoped**)

The traffic cops (**EmpController**)

The strict contracts (**IEmployee**)

The heavy lifting (**EmployeeService**)

The translator (**EmpContext** & **Models**)

Run the program. It executes identically to tightly-coupled code, but now your API is testable,
Run the program. It executes identically to tightly-coupled code, but now your API is testable,
maintainable, and built to scale infinitely.

NotebookLM